

DRDO • CFEES | INTERNSHIP PROJECT PRESENTATION

Employee Attrition Analytics System

A Comprehensive Pipeline for Workforce Ingestion, Database Storage, & Visualization

Pukhraj Singh & Yuvraj Punia
Role: Interns @ DRDO – CFEES

PROJECT OVERVIEW

Objective

Developed an end-to-end analytical data pipeline designed to parse unstructured organizational information, map it to structured storage environments, and isolate retention risks using multidimensional visualizations.

- ✓ **CSV to MySQL Data Import:** Safe parsing and insertion of 4,653 records from raw files into relational tables.
- ✓ **Database Management:** Using MySQL to organize employee records and carry out structured data operations.
- ✓ **SQL-Based Analytics:** Running performance optimization queries to analyze attrition patterns and workforce trends.
- ✓ **Attrition Dashboard:** Generating an automated multi-chart analytics dashboard via Matplotlib.



Technologies Used

Python, MySQL, Pandas, Matplotlib, SQL



Dataset Parameters

Total Records: 4,653 Employees

Source: Employee Attrition Dataset

Output Asset: attrition_dashboard.png

PROJECT STRUCTURE & ENVIRONMENT

Directory Tree Map

```
Employee-Analysis/  
├── Employee.csv      # Source raw data records  
├── import_csv.py     # Ingestion data script  
├── analytics.py      # Query & dashboard compiler  
├── attrition_dashboard.png # Exported visualization png  
├── requirements.txt  # Project environment locks  
└── README.md        # Documentation file
```

Environment Configuration

The runtime environment relies on the following structural system dependency extensions mapped within `requirements.txt`:

System Requirements

- `matplotlib`
- `mysql-connector-python`

Execution Steps

```
# 1. Install Dependencies  
pip install -r requirements.txt  
  
# 2. Ingest Data into MySQL  
python import_csv.py  
  
# 3. Generate Visual Dashboard  
python analytics.py
```

DATASET ATTRIBUTES SCHEMA

The analytics system maps and processes the following relational data columns:

Column Name	Analytical Type / Description	Relational Database Mapping
Education	Scholastic credentials qualification levels	VARCHAR(100)
JoiningYear	Temporal baseline for cohort identification	INT
City	Geographical operational facility nodes	VARCHAR(100)
PaymentTier	Salary classification distribution index (1-3)	INT
Age	Demographic parameter vector	INT
Gender	Demographic classification vector	VARCHAR(20)
EverBenched	Operational utilization status flag (Yes/No)	VARCHAR(10)
ExperienceInCurrentDomain	Tenure depth index inside specific functional role	INT
LeaveOrNot	Target vector matrix status indicator (0=Stay, 1=Left)	INT

DATA INGESTION IMPLEMENTATION (PART 1)

```
import sys
import mysql.connector
import pandas as pd

print("All libraries loaded.", flush=True)

df = pd.read_csv("Employee.csv")
df.columns = df.columns.str.strip()

print(f"CSV loaded: {len(df)} rows", flush=True)
print(df.head(2).to_string(), flush=True)

conn = mysql.connector.connect(
    host="127.0.0.1",
    port=3306,
    user="root",
    password="Ctmz2020@",
    database="employee_db",
    connection_timeout=30
)
cursor = conn.cursor()
print("Connected to MySQL.", flush=True)

# Clear any previous test data first
cursor.execute("TRUNCATE TABLE employees")
conn.commit()
print("Table cleared.", flush=True)
```

Console Execution Output

```
All libraries loaded.
CSV loaded: 4653 rows
  Education  JoiningYear  City  ...  LeaveOrNot
0 Bachelors      2017  Bangalore  ...         0
1   Masters      2015    Pune    ...         1

Connected to MySQL.
Table cleared.
```

The script builds runtime handshakes, cleans text spacing artifacts on columns, logs diagnostics, and empties target database entities before updates.

DATA INGESTION IMPLEMENTATION (PART 2)

```
# Build all rows as a list of tuples
rows = [
    (
        str(row["Education"]),
        int(row["JoiningYear"]),
        str(row["City"]),
        int(row["PaymentTier"]),
        int(row["Age"]),
        str(row["Gender"]),
        str(row["EverBenched"]),
        int(row["ExperienceInCurrentDomain"]),
        int(row["LeaveOrNot"])
    )
    for _, row in df.iterrows()
]

print(f"Rows prepared: {len(rows)}", flush=True)

cursor.executemany("""
    INSERT INTO employees
    (Education, JoiningYear, City, PaymentTier, Age,
     Gender, EverBenched, ExperienceInCurrentDomain, LeaveOrNot)
    VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
""", rows)

conn.commit()
print(f"Done. Rows inserted: {cursor.rowcount}", flush=True)

cursor.close()
conn.close()
```

Console Performance Output

```
Rows prepared: 4653
Done. Rows inserted: 4653
```

Technical Design Highlights

- ✓ **Primitive Explicit Casting:** Maps pandas column variants into safe native Python types to bypass data mismatch runtime blocks.
- ✓ **Executemany Multi-Row Driver:** Bundles the 4,653 dataset objects into an optimized transaction pipeline rather than opening standalone insert requests.

DASHBOARD SCRIPT: INITIALIZATION & LAYOUT

```
import sys
import mysql.connector
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

print("Python:", sys.executable, flush=True)
print("Starting Dashboard...", flush=True)

try:
    conn = mysql.connector.connect(
        host="127.0.0.1", port=3306, user="root",
        password="Ctmz2020@", database="employee_db", connection_timeout=30
    )
    print("Connected to MySQL", flush=True)
    cursor = conn.cursor()

    fig = plt.figure(figsize=(18, 12))
    fig.suptitle(
        "Employee Attrition Analytics Dashboard",
        fontsize=20, fontweight="bold", y=0.98
    )

    # Defining a structured 2-row, 3-column sub-plot grid canvas
    gs = gridspec.GridSpec(2, 3, figure=fig, hspace=0.45, wspace=0.35)
```

Pipeline Setup Logs

```
Python: C:\Python311\python.exe
Starting Dashboard...
Connected to MySQL
```

Architectural Framework

Initializes a fixed database connection array and uses Matplotlib's advanced GridSpec framework engine layout to assign geometry metrics for the dashboard plots.

DASHBOARD SCRIPT: CHARTS 1, 2 & 3

```
# Chart 1 - Overall Attrition Pie Chart
cursor.execute("SELECT LeaveOrNot, COUNT(*) FROM employees GROUP BY LeaveOrNot")
data = dict(cursor.fetchall())
ax1 = fig.add_subplot(gs[0, 0])
ax1.pie([data.get(0, 0), data.get(1, 0)], labels=["Stayed", "Left"], autopct="%1.1f%%", startangle=90)
ax1.set_title("Overall Attrition Rate")
print("Chart 1 Complete", flush=True)

# Chart 2 - Attrition by City Vertical Bar
cursor.execute("SELECT City, ROUND(SUM(LeaveOrNot)/COUNT(*)*100,1) FROM employees GROUP BY City ORDER BY City")
rows = cursor.fetchall()
ax2 = fig.add_subplot(gs[0, 1])
ax2.bar([r[0] for r in rows], [r[1] for r in rows])
ax2.set_title("Attrition by City")
ax2.set_ylabel("Attrition %")
print("Chart 2 Complete", flush=True)

# Chart 3 - Attrition by Education Horizontal Bar
cursor.execute("SELECT Education, ROUND(SUM(LeaveOrNot)/COUNT(*)*100,1) FROM employees GROUP BY Education ORDER BY Education")
rows = cursor.fetchall()
ax3 = fig.add_subplot(gs[0, 2])
ax3.barh([r[0] for r in rows], [r[1] for r in rows])
ax3.set_title("Attrition by Education")
print("Chart 3 Complete", flush=True)
```

Subplot Status Logs

```
Chart 1 Complete
Chart 2 Complete
Chart 3 Complete
```

Analytical Pipeline

Runs parallel relational SQL groupings across the top axis row to resolve organizational ratios, geographic regional vectors, and degree distribution variables.

DASHBOARD SCRIPT: CHARTS 4, 5 & 6

```
# Chart 4 - Attrition by Gender Bar Chart
cursor.execute("SELECT Gender, ROUND(SUM(LeaveOrNot)/COUNT(*)*100,1) FROM employees GROUP BY Gender")
rows = cursor.fetchall()
ax4 = fig.add_subplot(gs[1, 0])
ax4.bar([r[0] for r in rows], [r[1] for r in rows])
ax4.set_title("Attrition by Gender")
print("Chart 4 Complete", flush=True)

# Chart 5 - Attrition by Payment Tier Discrete Bar
cursor.execute("SELECT PaymentTier, ROUND(SUM(LeaveOrNot)/COUNT(*)*100,1) FROM employees GROUP BY PaymentTier ORDER BY PaymentTier")
rows = cursor.fetchall()
ax5 = fig.add_subplot(gs[1, 1])
ax5.bar([str(r[0]) for r in rows], [r[1] for r in rows])
ax5.set_title("Attrition by Payment Tier")
print("Chart 5 Complete", flush=True)

# Chart 6 - Attrition trend by Experience Line Plot
cursor.execute("SELECT ExperienceInCurrentDomain, ROUND(SUM(LeaveOrNot)/COUNT(*)*100,1) FROM employees GROUP BY ExperienceInCurrentDomain ORDER BY ExperienceInCurrentDomain")
rows = cursor.fetchall()
ax6 = fig.add_subplot(gs[1, 2])
ax6.plot([r[0] for r in rows], [r[1] for r in rows], marker="o")
ax6.set_title("Attrition by Experience")
print("Chart 6 Complete", flush=True)
```

Matrix Thread Logs

Chart 4
Complete
Chart 5
Complete
Chart 6
Complete

Analytical Pipeline

Maps
secondary grid
rows containing
gender vectors,
structural
organizational
compensation
level
distributions,
and current
field experience
line
coordinates.

DASHBOARD SCRIPT: COMPILED & INTERCEPT

```
# -----  
plt.savefig("attrition_dashboard.png", dpi=150, bbox_inches="tight")  
print("Dashboard saved as attrition_dashboard.png", flush=True)  
  
cursor.close()  
conn.close()  
print("SUCCESS", flush=True)  
  
except Exception as e:  
    print("ERROR:", flush=True)  
    print(type(e).__name__, flush=True)  
    print(e, flush=True)
```

Termination Diagnostics

Dashboard saved as attrition_dashboard.png
SUCCESS

Process Conclusion

Safely packages vector graph calculations into high DPI physical assets, terminates open database ports and integrates a bulletproof global Exception intercept layer framework.

GENERATED ATTRITION DASHBOARD OUTPUT MATRIX

Visualizing full consolidated matrix dashboard generated natively inside the project folder tree:



PASTE FULL ATTRITION DASHBOARD GRID IMAGE HERE

DASHBOARD BREAKDOWN: SUBPLOTS 1, 2 & 3



PASTE CHART 1 HERE



PASTE CHART 2 HERE



PASTE CHART 3 HERE

Chart 1: Overall Attrition

Establishes a **34.4% turnover rate** across 4,653 personnel records, highlighting a critical organizational challenge.

Chart 2: City Analysis

Isolates a major regional retention risk spike centered within the **Pune operational facility (~50.1%)**.

Chart 3: Education Variable

Reveals that **Masters degree holders possess the highest exit rate (~47.7%)** compared to other tiers.

DASHBOARD BREAKDOWN: SUBPLOTS 4, 5 & 6



PASTE CHART 4 HERE



PASTE CHART 5 HERE



PASTE CHART 6 HERE

Chart 4: Gender Analysis

Highlights a clear gender risk disparity: **Female employees show higher attrition (~45.9%)** than Male cohorts.

Chart 5: Compensation Tiers

Identifies an anomaly: **Tier 2 (mid-pay level) employees suffer an aggressive attrition spike (~59.9%)**.

Chart 6: Experience Tenure

Maps a sharp drop in attrition reaching its lowest at 6 years (~25.0%) before increasing at year 7 (~33.3%).

FOCUS: WORKFORCE AGE DISTRIBUTION



Workforce Histogram Space

The dashed framework on the right marks the dedicated presentation container to display your generated workforce age frequency distribution profile.



PASTE WORKFORCE AGE HISTOGRAM HERE

DEMOGRAPHIC INSIGHTS MATRIX

Key Demographic Observations

- ✓ **Young Workforce Concentration:** A massive density spike is observed between ages 26–28 (~640 active personnel).
- ✓ **The Age 30 Retention Drop:** Staff counts drop off sharply after age 30, highlighting a critical organizational pivot point.
- ✓ **Entry-Level Pipeline:** The lowest baseline presence is found within the ages 22–24 segment (~50 personnel).
- ✓ **Summary Impact:** The workforce leans heavily toward early-career personnel, making it highly susceptible to volatile market trends.

Pipeline Data Volume Vector

Calculated mean workforce baseline age settles precisely at 29.4 years across the aggregated transaction table rows.

Retention Action Item

Early career mentorship programs and localized incentive adjustments should target the 25–29 cohort to address high entry-level turnover.

RESULTS & CONCLUSION

Quantifiable Project Milestones

- ✓ Successfully processed and imported 4,653 complete employee records into MySQL database architecture.
- ✓ Executed optimized, SQL-driven group filtering analysis on multi-dimensional employee attrition patterns.
- ✓ Generated an automated multi-chart analytics dashboard output file asset safely saved to disk space.

Project Significance Summary

The analytics engine successfully converts raw workforce rows into actionable strategic insights. By connecting Python data libraries directly to relational database systems, it provides leadership clear data to tackle attrition bottlenecks systematically.

THANK YOU

Questions & Discussion Portfolio

Yuvraj Punia & Pukhraj Singh

Interns @ DRDO – CFEES

Project: Employee Data Analysis & Visualization Infrastructure
